

Week_9-10_Assignment

Velazquez-Feliciano, Adrian

DSC 640: Data Presentation and Visualization

03/01/2026

TSA Complaints Analysis - Summary Paper

AUDIENCE

The intended audience for this analysis is TSA leadership and airport operations managers - professionals who are familiar with security screening processes and operational data but may not be data scientists. They understand terms like PreCheck, checkpoint screening, and baggage handling, so moderate data literacy is assumed. The visuals are designed to be clear and actionable without requiring deep statistical knowledge.

PURPOSE & CALL TO ACTION

The purpose of this story is to highlight where TSA complaint volumes are highest, which categories drive the most dissatisfaction, and how the COVID-19 pandemic disrupted and subsequently amplified complaint trends.

The call to action is directed at TSA leadership:

Prioritize resources and process improvements at the top-complaint airports (LAX, JFK, EWR) and address the dominant complaint category - the Expedited Passenger Screening Program (PreCheck/EPSP) - which accounts for over 569,000 complaints, nearly 4x any other category.

Specifically, TSA should audit PreCheck Lane availability and staffing at high-volume airports and investigate why complaints surged post-COVID despite lower passenger volumes in 2020-2021.

MEDIUM

The medium chosen is a Python-generated infographic-style visual report (PDF/PNG exports), suitable for inclusion in an executive briefing or operational review deck. This medium was selected because it allows precise control over layout, color, and annotation, and can be easily embedded in PowerPoint or Word for distribution.

DESIGN CHOICES

A consistent color palette of TSA Blue (#003087) and TSA Red (#CC0000) was used throughout to reinforce brand identity and create visual cohesion. Seven visuals were created:

1. Line chart - shows the overall complaint trend and COVID impact over time.
2. Horizontal bar chart - ranks the top 10 complaint categories for quick comparison.
3. Heatmap - reveals seasonal and year-over-year patterns simultaneously.
4. Box plot - shows the distribution and variability of monthly complaints at top airports.
5. Choropleth map - provides a spatial view of complaint concentration by state.
6. Stacked bar chart - illustrates how complaint categories shifted during COVID years.
1. Custom bubble chart - a novel view combining complaint volume with category diversity per airport, revealing which airports have both high volume AND broad complaint types.

ETHICAL CONSIDERATIONS

The data is presented without identifying individual passengers or TSA officers, protecting privacy. Complaint counts are presented as raw totals rather than rates per passenger, which could be misleading - a limitation acknowledged here. No data was excluded or cherry-picked; all years and all airports in the dataset are represented. The COVID annotation is included to provide context rather than to excuse performance gaps, ensuring the audience interprets the 2020 dip accurately.

Week_9-10_Assignment

Velazquez-Feliciano, Adrian

DSC 640: Data Presentation and Visualization

03/01/2026

```
In [6]:
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import seaborn as sns
import geopandas as gpd
import warnings
import os as os

warnings.filterwarnings("ignore")
# Set the directory
directory = r"C:\Users\velaz\OneDrive\Desktop\DSC640-2026"
os.chdir(directory)

warnings.filterwarnings('ignore')
# =====
# 1. LOAD DATA
# =====
print("Loading datasets...")
df_airport = pd.read_csv("complaints-by-airport.csv", encoding="ascii")
df_category = pd.read_csv("complaints-by-category.csv", encoding="ascii")
df_subcategory = pd.read_csv("complaints-by-subcategory.csv", encoding="ascii")
df_iata = pd.read_csv("iata-icao.csv", encoding="utf-8")

for df in [df_airport, df_category, df_subcategory]:
    df["year"] = df["year_month"].str[:4].astype(int)
    df["month"] = df["year_month"].str[5:7].astype(int)

# US airports with geo info
df_iata_us = df_iata[df_iata["country_code"] == "US"][
    ["iata", "airport", "latitude", "longitude", "region_name"]].copy()
df_iata_us.columns = ["airport_code", "airport_name", "latitude", "longitude", "state"]

airport_totals = df_airport.groupby("airport")["count"].sum().reset_index()
airport_totals.columns = ["airport_code", "total_complaints"]
airport_totals = airport_totals.merge(df_iata_us, on="airport_code", how="left")
airport_totals = airport_totals.sort_values("total_complaints", ascending=False)

print("Top airport: " + airport_totals.iloc[0]["airport_code"])

# =====
# 2. SHARED PREP
# =====
TSA_BLUE = "#003087"; TSA_RED = "#CC0000"; TSA_GOLD = "#FFB81C"

yearly = df_airport[df_airport["year"] < 2024].groupby("year")["count"].sum().reset_index()

top10_cats = (df_category.groupby("clean_cat")["count"].sum()
              .sort_values(ascending=False).head(10).reset_index())
top10_cats.columns = ["category", "count"]

monthly_hm = df_airport[df_airport["year"] < 2024].groupby(["year", "month"])["count"].sum().reset_index()
hm_pivot = monthly_hm.pivot(index="year", columns="month", values="count")
hm_pivot.columns = ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"]

top5_cats = top10_cats.head(5)["category"].tolist()
covid_cats = df_category[df_category["clean_cat"].isin(top5_cats)].copy()
covid_yearly = (covid_cats[covid_cats["year"].isin([2019, 2020, 2021, 2022, 2023])]
               .groupby(["year", "clean_cat"])["count"].sum().reset_index())
pivot_covid = covid_yearly.pivot(index="year", columns="clean_cat", values="count").fillna(0)
short_names = {c: c.split(" ")[:-1] if len(c) > 20 else c for c in pivot_covid.columns}
pivot_covid.columns = [short_names[c] for c in pivot_covid.columns]

cat_diversity = (df_category[df_category["count"] > 0]
               .groupby("airport")["clean_cat"].nunique().reset_index())
cat_diversity.columns = ["airport_code", "cat_diversity"]
bubble_data = airport_totals.head(30).merge(cat_diversity, on="airport_code", how="left").dropna()

# =====
# 3. VISUAL 1 - Line Chart: Yearly Trend
# =====
print("Generating Visual 1: Yearly Trend...")
fig, ax = plt.subplots(figsize=(10, 4))
ax.fill_between(yearly["year"], yearly["count"], alpha=0.12, color=TSA_BLUE)
ax.plot(yearly["year"], yearly["count"], color=TSA_BLUE, linewidth=2.5, marker="o",
        markersize=8, markerfacecolor=TSA_RED, markeredgcolor="white", markeredgewidth=1.5)
ax.annotate("COVID-19 \ Drop",
            xy=(2020, yearly[yearly["year"]==2020]["count"].values[0]),
            xytext=(2020.3, 80000),
            arrowprops=dict(arrowstyle=">", color=TSA_RED, lw=1.5),
            color=TSA_RED, fontsize=9, fontweight="bold")
ax.set_title("TSA Complaints Over Time (2015-2023)", fontsize=14, fontweight="bold", color=TSA_BLUE, pad=12)
ax.set_xlabel("Year", fontsize=10); ax.set_ylabel("Total Complaints", color="black", fontsize=10)
ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda x, _ : f"{int(x):,}"))
ax.set_facecolor("#F8F9FC"); fig.patch.set_facecolor("white")
ax.grid(axis="y", alpha=0.3, linestyle="--")
sns.despine()
plt.tight_layout()
plt.savefig("v1_trend.png", dpi=150, bbox_inches="tight")
plt.show()
print(" Saved: v1_trend.png")

# =====
# 4. VISUAL 2 - Horizontal Bar: Top 10 Categories
# =====
print("Generating Visual 2: Top 10 Categories...")
top10_sorted = top10_cats.sort_values("count")
bar_colors = [TSA_RED if i >= 8 else TSA_BLUE for i in range(len(top10_sorted))]

fig, ax = plt.subplots(figsize=(10, 5))
bars = ax.barh(top10_sorted["category"], top10_sorted["count"],
               color=bar_colors, edgecolor="white", height=0.65)
for bar, val in zip(bars, top10_sorted["count"]):
    ax.text(bar.get_width() + 4000, bar.get_y() + bar.get_height()/2,
            f"{int(val):,}", va="center", fontsize=9, color="#333")
ax.set_title("Top 10 Complaint Categories (All Years)", fontsize=14, fontweight="bold", color=TSA_BLUE, pad=12)
ax.set_xlabel("Total Complaints", fontsize=10)
ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda x, _ : f"{int(x):,}"))
ax.set_facecolor("#F8F9FC"); fig.patch.set_facecolor("white")
ax.grid(axis="x", alpha=0.3, linestyle="--")
sns.despine()
plt.tight_layout()
plt.savefig("v2_categories.png", dpi=150, bbox_inches="tight")
plt.show()
print(" Saved: v2_categories.png")

# =====
# 5. VISUAL 3 - Heatmap: Year x Month
# =====
print("Generating Visual 3: Heatmap...")
fig, ax = plt.subplots(figsize=(12, 5))
sns.heatmap(hm_pivot, annot=True, fmt=".0f", cmap="YlOrRd", linewidths=0.5,
            linecolor="white", ax=ax, cbar_kws={"label": "Complaints"})
ax.set_title("Complaint Volume Heatmap: Year x Month", fontsize=14, fontweight="bold", color=TSA_BLUE, pad=12)
ax.set_xlabel("Month", fontsize=10); ax.set_ylabel("Year", fontsize=10)
plt.tight_layout()
plt.savefig("v3_heatmap.png", dpi=150, bbox_inches="tight")
plt.show()
print(" Saved: v3_heatmap.png")

# =====
# 6. VISUAL 4 - Box Plot: Monthly Distribution Top 10 Airports
# =====
print("Generating Visual 4: Box Plot...")
box_order = airport_totals.head(10)["airport_code"].tolist()
monthly_per_airport = (df_airport[df_airport["airport"].isin(box_order)]
                     .groupby(["airport", "year_month"])["count"].sum().reset_index())
bp_data = [monthly_per_airport[monthly_per_airport["airport"] == code]["count"].values
           for code in box_order]

fig, ax = plt.subplots(figsize=(12, 5))
ax.boxplot(bp_data, labels=box_order, patch_artist=True, notch=False,
            medianprops=dict(color=TSA_RED, linewidth=2),
            boxprops=dict(facecolor="#D6E4F7", color=TSA_BLUE),
            whiskerprops=dict(color=TSA_BLUE), capprops=dict(color=TSA_BLUE),
            flierprops=dict(marker="o", color=TSA_RED, alpha=0.4, markersize=4))
ax.set_title("Monthly Complaint Distribution: Top 10 Airports", fontsize=14, fontweight="bold", color=TSA_BLUE, pad=12)
ax.set_xlabel("Airport Code", fontsize=10); ax.set_ylabel("Monthly Complaints", fontsize=10)
ax.set_facecolor("#F8F9FC"); fig.patch.set_facecolor("white")
ax.grid(axis="y", alpha=0.3, linestyle="--")
sns.despine()
plt.tight_layout()
plt.savefig("v4_boxplot.png", dpi=150, bbox_inches="tight")
plt.show()
print(" Saved: v4_boxplot.png")

# =====
# 7. VISUAL 5 - Choropleth Map: Complaints by State
# =====
print("Generating Visual 5: Choropleth Map...")
state_totals = (airport_totals.dropna(subset=["state"])
               .groupby("state")["total_complaints"].sum().reset_index())
us_states = gpd.read_file(
    "https://raw.githubusercontent.com/PublicaMundi/MappingAPI/master/data/geojson/us-states.json")
us_states = us_states.rename(columns={"name": "state"})
us_states = us_states.merge(state_totals, on="state", how="left")
us_states["total_complaints"] = us_states["total_complaints"].fillna(0)

fig, ax = plt.subplots(figsize=(14, 7))
us_states.plot(column="total_complaints", cmap="YlOrRd", linewidth=0.5, edgecolor="white",
               legend=True, ax=ax,
               legend_kws={"label": "Total Complaints", "orientation": "horizontal",
                           "shrink": 0.5, "pad": 0.02})
ax.set_xlim(-130, -65); ax.set_ylim(23, 52)
ax.set_title("TSA Complaints by State (Choropleth)", fontsize=14, fontweight="bold", color=TSA_BLUE, pad=12)
ax.axis("off"); fig.patch.set_facecolor("white")
plt.tight_layout()
plt.savefig("v5_choropleth.png", dpi=150, bbox_inches="tight")
plt.show()
print(" Saved: v5_choropleth.png")

# =====
# 8. VISUAL 6 - Stacked Bar: COVID Category Impact
# =====
print("Generating Visual 6: COVID Stacked Bar...")
colors_stack = [TSA_BLUE, TSA_RED, TSA_GOLD, "#2E8B57", "#8B008B"]

fig, ax = plt.subplots(figsize=(10, 5))
pivot_covid_pivot = pivot_covid.pivot(index="year", columns="clean_cat", values="count")
ax.set_title("Top 5 Complaint Categories: COVID Impact (2019-2023)",
            fontsize=14, fontweight="bold", color=TSA_BLUE, pad=12)
ax.set_xlabel("Year", fontsize=10); ax.set_ylabel("Total Complaints", fontsize=10)
ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda x, _ : f"{int(x):,}"))
ax.legend(loc="upper right", fontsize=8, framealpha=0.8, title="Category")
ax.set_facecolor("#F8F9FC"); fig.patch.set_facecolor("white")
plt.xticks(rotation=0)
ax.grid(axis="y", alpha=0.3, linestyle="--")
sns.despine()
plt.tight_layout()
plt.savefig("v6_covid_stacked.png", dpi=150, bbox_inches="tight")
plt.show()
```

Loading datasets...

Top airport: LAX

Generating Visual 1: Yearly Trend...



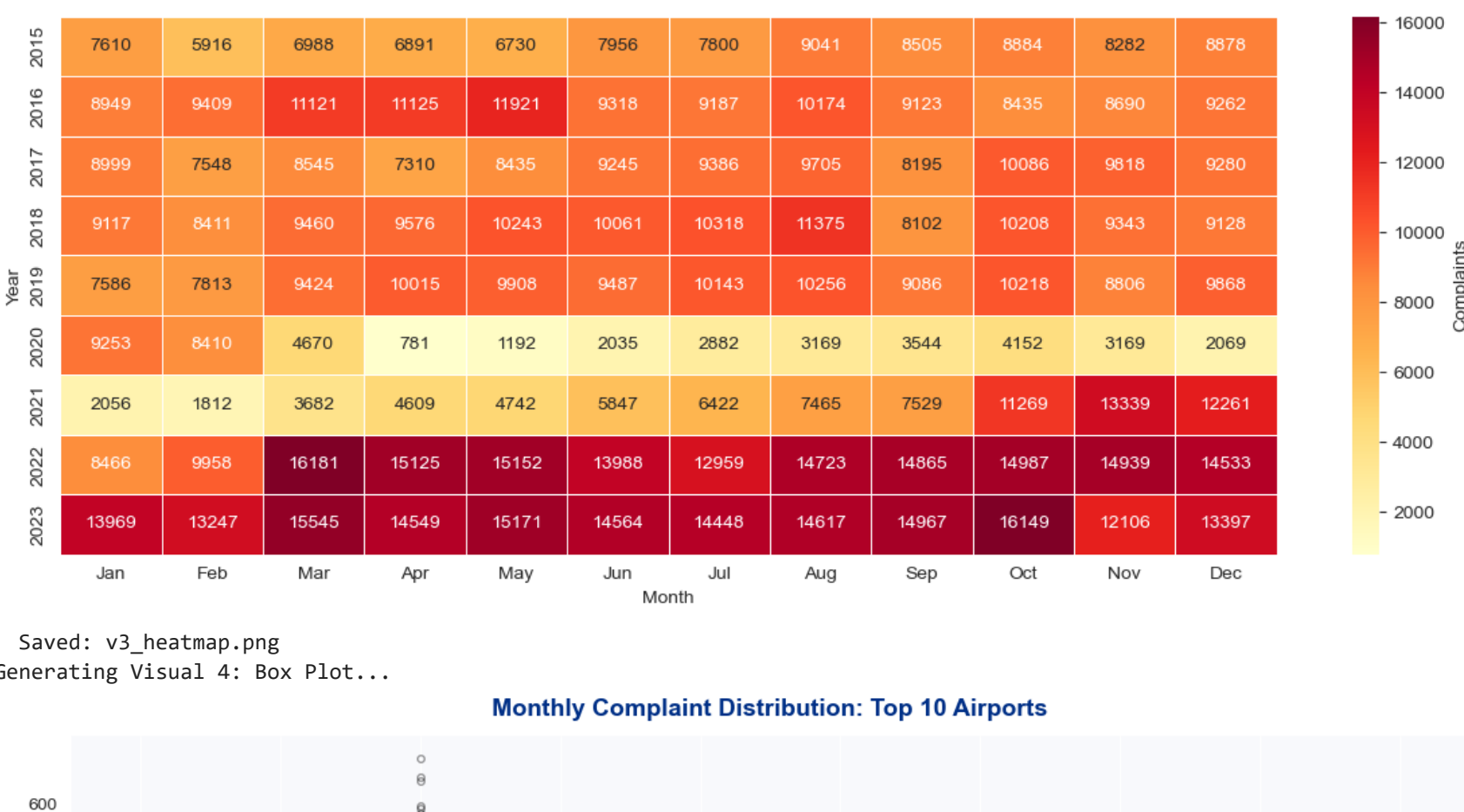
Saved: v1_trend.png

Generating Visual 2: Top 10 Categories...



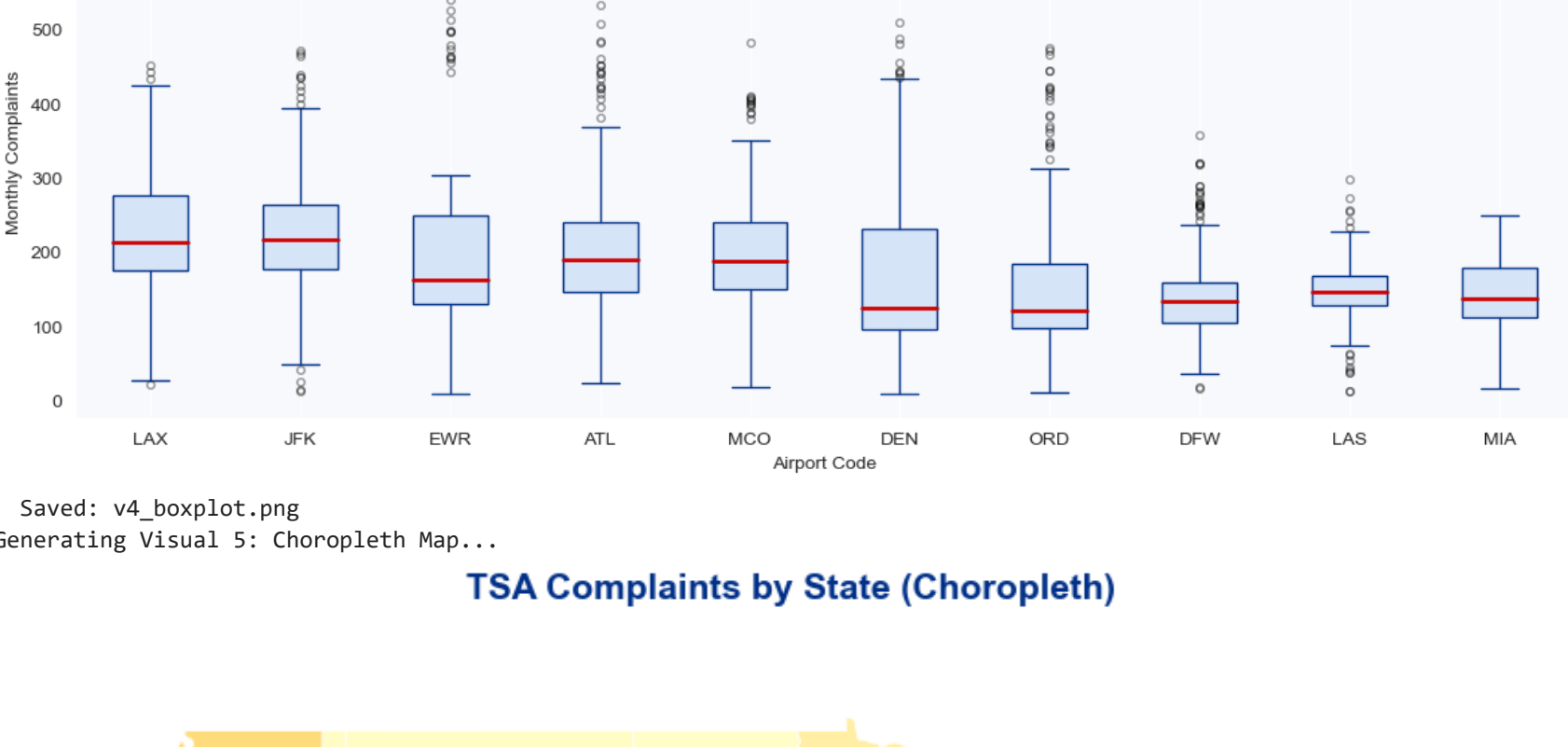
Saved: v2_categories.png

Generating Visual 3: Heatmap...



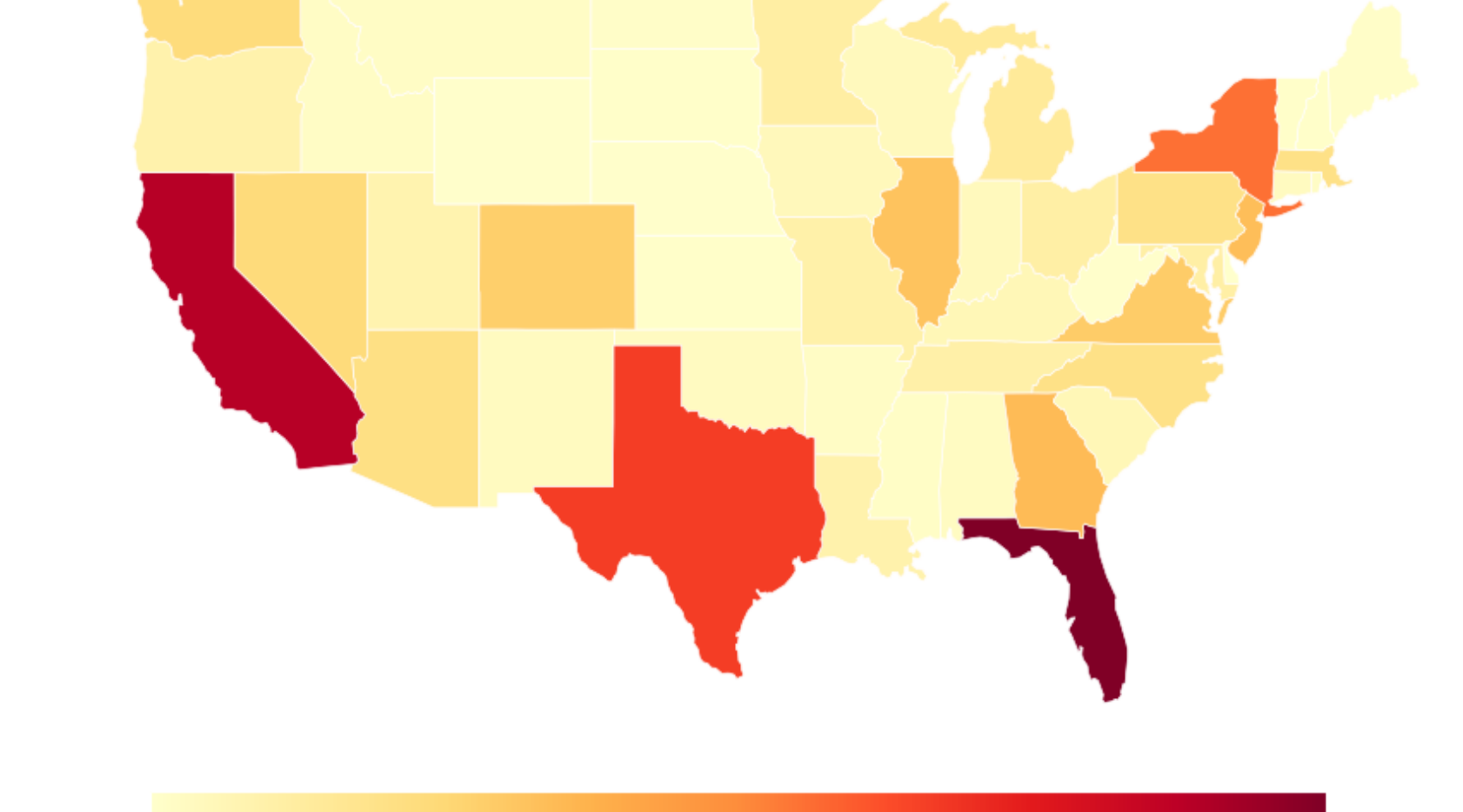
Saved: v3_heatmap.png

Generating Visual 4: Box Plot...



Saved: v4_boxplot.png

Generating Visual 5: Choropleth Map...



Saved: v5_choropleth.png

Generating Visual 6: COVID Stacked Bar...

